Guia Prático: Protegendo nossos Objetos com Encapsulamento

Até agora, aprendemos que uma **Classe** serve para agrupar os dados (atributos) e as ações (métodos) de um objeto em um só lugar. Isso já deixou nosso código muito mais organizado!

Porém, no mundo real da programação, existe um perigo oculto quando deixamos nossos dados "soltos". Hoje, vamos aprender a técnica de **Encapsulamento** para resolver esse problema.

1. O Problema: Por que precisamos proteger nossos dados?

Imagine o nosso sistema que calcula a área e o preço de um `Terreno`. No nosso primeiro código, as variáveis estavam públicas, ou seja, qualquer um podia alterá-las no fluxo principal do programa.

Veja o que um usuário descuidado (ou mal-intencionado) poderia fazer:

```
meu_terreno = Terreno(10, 20, 150)

# O usuário altera a largura diretamente para um número impossível:
meu_terreno.largura = -50
```

Pronto, o sistema quebrou a lógica! Não existe terreno com largura negativa, mas o Python aceitou porque a variável estava **desprotegida**. O cálculo da área será negativo, o preço será negativo e o programa perderá totalmente o sentido.

2. O que é Encapsulamento?

A palavra **Cápsula** lembra um remédio. O pó do remédio não fica solto na sua mão; ele fica protegido dentro de uma cápsula de gelatina.

Na programação, **Encapsular** significa "esconder" ou "trancar" os dados sensíveis dentro do objeto, para que o programa principal não possa mexer neles diretamente de forma errada.

Para interagir com esses dados escondidos, a própria classe fornece "portas seguras" (métodos) que possuem regras de validação.

3. Como fazer o Encapsulamento no Python (Passo a Passo)

Vamos transformar o nosso terreno comum em um "terreno blindado". Siga os passos abaixo para entender cada comando novo.

Passo 1: Trancando os atributos no Cofre (O `__`)

A primeira coisa que fazemos é avisar ao Python que nossos atributos são **Privados** (secretos). Fazemos isso colocando dois sublinhados (`__`) antes do nome da variável dentro do `__init__`.

```
class Terreno:
    def __init__(self, largura, comprimento, valor_metro_quadrado):
        # O duplo sublinhado (  ) esconde a variável.
        self.  largura = largura
        self.  comprimento = comprimento
        self.__valor_metro_quadrado = valor_metro_quadrado
```

O que acontece agora? Se alguém tentar fazer `meu terreno.  largura = 100` lá no final do código, o Python vai ignorar. A variável original está segura dentro do objeto.

Passo 2: Criando a "Janela de Leitura" (Os Getters e o `@property`)

Se trancamos a largura no cofre, como vamos saber o tamanho do terreno depois? Precisamos criar uma forma segura de apenas **LER** esse dado.

No Python, usamos um decorador chamado `@property` . Ele transforma um método comum em uma "propriedade" que pode ser lida facilmente.

```
    @property
    def largura(self):
        return self.__largura
```

A intenção: Quando o usuário digitar `print(meu terreno.largura)` , o Python, por baixo dos panos, vai acionar esse método, entrar no cofre, pegar o valor de `self.__largura` e devolver para a tela. O usuário apenas olha, mas não toca!

Passo 3: Criando a "Porta de Alteração" (Os Setters)

E se o usuário digitar errado e precisar corrigir a largura? Não podemos deixar ele alterar diretamente. Criamos um **Setter** (um método de configuração).

Aqui é onde a mágica acontece: nós colocamos um segurança (um `if`) na porta!

```
    @largura.setter
    def largura(self, novo_valor):
        if novo valor > 0:
            self.__largura = novo_valor # Permite a mudança
        else:
            print("Erro: A largura deve ser maior que zero!") # Bloqueia a mudança
```

A intenção: Quando o usuário tentar fazer `meu terreno.largura = -10` , o Python percebe o sinal de igual (`=`) e manda o `-10` para a variável `novo_valor` . O nosso `if` barra o valor negativo e protege a integridade do terreno.

Passo 4: Atualizando os Cálculos Internos

Como os métodos `calcular_area` e `calcular_preco` estão **dentro** da classe, eles têm passe livre para acessar o cofre diretamente usando os dois sublinhados.

```
    def calcular_area(self):
        # Aqui dentro, nós usamos os atributos com __ normalmente
        return self.__largura * self.__comprimento
```

4. O Resultado no Fluxo Principal

A melhor parte do encapsulamento em Python é que, para quem está usando a classe, nada parece ter mudado. O código principal continua limpo e elegante.

```
largura_digitada = float(input("Digite a largura do terreno: "))
comprimento_digitado = float(input("Digite o comprimento do terreno: "))
valor_digitado = float(input("Digite o valor do metro quadrado: "))

# Instanciando o objeto
meu_terreno = Terreno(largura_digitada, comprimento_digitado, valor_digitado)

# TENTATIVA INVÁLIDA
meu_terreno.largura = -50
# Ao tentar fazer isso, o programa NÃO quebra! Ele apenas imprime a mensagem de Err

meu_terreno.mostrar_resultados()
```

🎯 Resumo da Aula

1. **Atributos Públicos** (`self.largura`): Qualquer um pode ler e alterar para valores incorretos.
2. **Atributos Privados** (`self.__largura`): Protegidos dentro da classe.
3. `@property` **(Getter)**: A maneira segura de **ler** o atributo secreto.
4. `@nome.setter` : A maneira segura de **alterar** o atributo secreto, passando por uma validação (`if/else`).